MOD -03 -> Advanced Java

LUMINAR
TECHNOLAB

Java Exception

Problem without exception handling

```
public class TryCatchExample {
    public static void main(String[] args) {
      int data=50/0; //may throw exception
      System.out.println("rest of the code");
   }
}
```

As displayed in the example, the rest of the code is not executed (in such case, the rest of the code statement is not printed).

There might be 100 lines of code after the exception. If the exception is not handled, all the code below the exception won't be executed.

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
```

Java Exception

What is Exception?

An exception is an event, which occurs during the execution of a program, that disrupts the normal flow of the program's instruction.

What is Exception Handling?

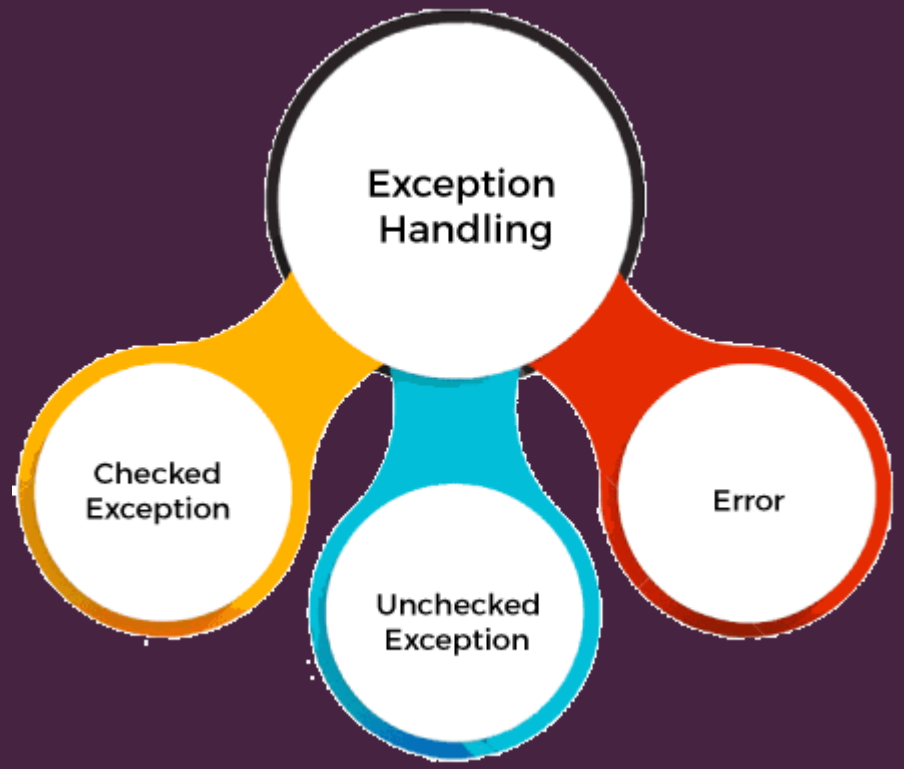
Exception handling is a mechanism to maintain the normal flow of the application.

An exception normally disrupts the normal flow of the application; that is why we need to handle exceptions.

Types of Java Exceptions

There are mainly two types of exceptions: checked and unchecked.

An error is considered as the unchecked exception.



Checked Exception

Checked exceptions are checked at compile-time

For example, IOException, SQLException, etc..

Unchecked Exception

Unchecked exceptions are not checked at compile-time, but they are checked at runtime.

For example, ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, etc.

Error

Error is irrecoverable. Some example of errors are OutOfMemoryError, VirtualMachineError, AssertionError etc.

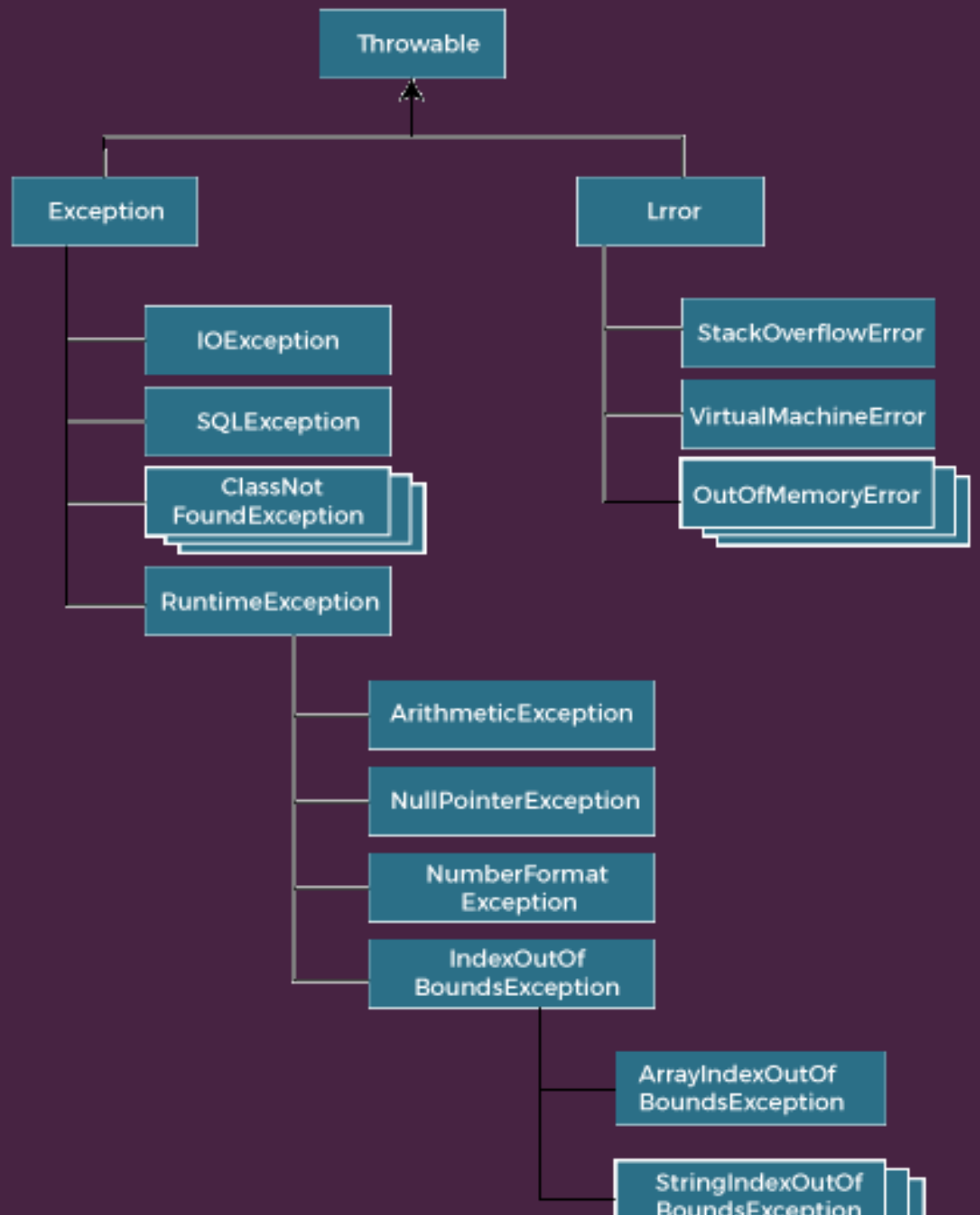
Checked exceptions (like IOException, SQLException) are enforced by the compiler, and must be handled. (i.e. Exception handling is primarily used to handle checked exceptions)

Unchecked exceptions (like NullPointerException, ArithmeticException) are runtime errors and usually arise from logic mistakes.

Saying it's the "programmer's fault" is true in principle but better phrased as "programming error" or "due to missing validation".

Hierarchy of Java Exception classes

The java.lang.Throwable class is the root class of Java Exception hierarchy inherited by two subclasses: Exception and Error.


Throwable
Exception
Lrror
IOException
SQLException
ClassNot FoundException
RuntimeException
StackOverflowError
VirtualMachineError
OutOfMemoryError
ArithmeticException
NullPointerException
NumberFormat Exception
IndexOutOf BoundsException
ArrayIndexOutOf BoundsException
StringIndexOutOf BoundsException

Java Exception Keywords

Keyword	Description
try	The "try" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block must be followed by either catch or finally.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
finally	The "finally" block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.
throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature.

Java try block

- Java try block is used to enclose the code that might throw an exception.
- It must be used within the method.
- If an exception occurs at the particular statement in the try block, the rest of the block code will not execute. So, it is recommended not to keep the code in try block that will not throw an exception.
- Java try block must be followed by either catch or finally block.

Java catch block

- Java catch block is used to handle the Exception by declaring the type of exception within the parameter.
- The declared exception must be the parent class exception (i.e., Exception) or the generated exception type.
- *However, the good approach is to declare the generated type of exception.*
- The catch block must be used after the try block only. You can use multiple catch block with a single try block.

Syntax of Java try-catch

```
try{
//code that may throw an exception
}catch(Exception_class_Name ref){}
```

Syntax of try-finally block

```
try{
//code that may throw an exception
}finally{}
```

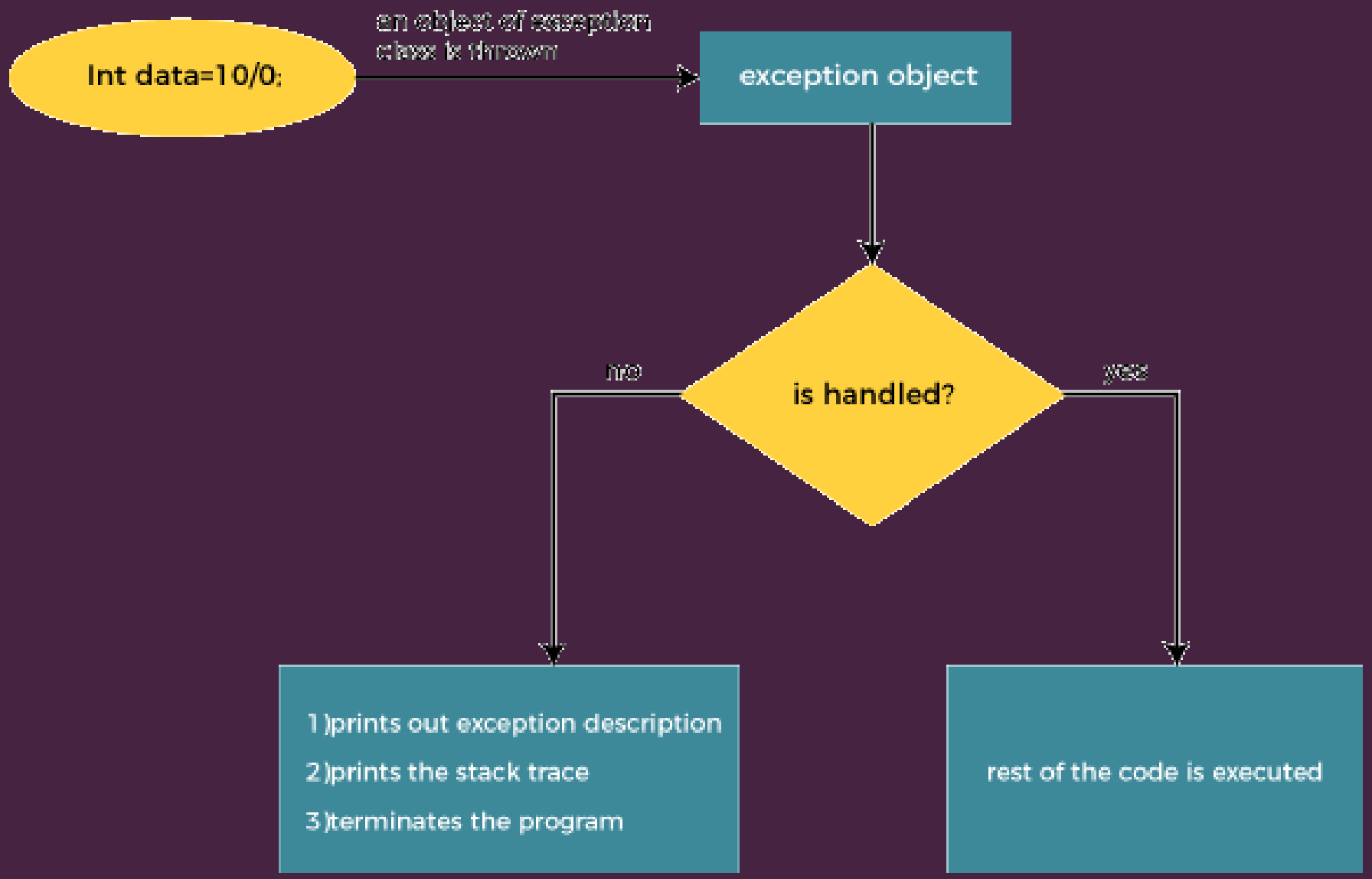
Demo Programs

Arithmetic Exception Demo

```
public class TryCatchExample{
 public static void main(String args[]){
  try{
    //code that may raise exception
    int data=10/0;
  }catch(ArithmeticException e){          // System.out.println("error..");
          e.printStackTrace();            //System.out.println(e);
}                                         //System.err.print("error");
  //rest code of the program
  System.out.println("rest of the code...");
 }
}
```

```
Exception in thread main java.lang.ArithmeticException:/ by zero
rest of the code...
```

Internal Working of Java try-catch block



The JVM firstly checks whether the exception is handled (by programmer)or not. If exception is not handled, JVM provides a default exception handler that performs the following tasks:

1. Prints out exception description.
2. Prints the stack trace (Hierarchy of methods where the exception occurred).
3. Causes the program to terminate.

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
```

But if the application programmer handles the exception, the normal flow of the application is maintained, i.e., rest of the code is executed.

ArrayIndex Exception Demo

```
public class TryCatchExample{
    public static void main(String[] args) {
        try {
            int arr[]= {1,3,5,7};
            System.out.println(arr[10]); //may throw exception
        } catch(ArrayIndexOutOfBoundsException e) {// handling the array exception
            System.out.println(e);
        }
        System.out.println("rest of the code");
    }

}
```

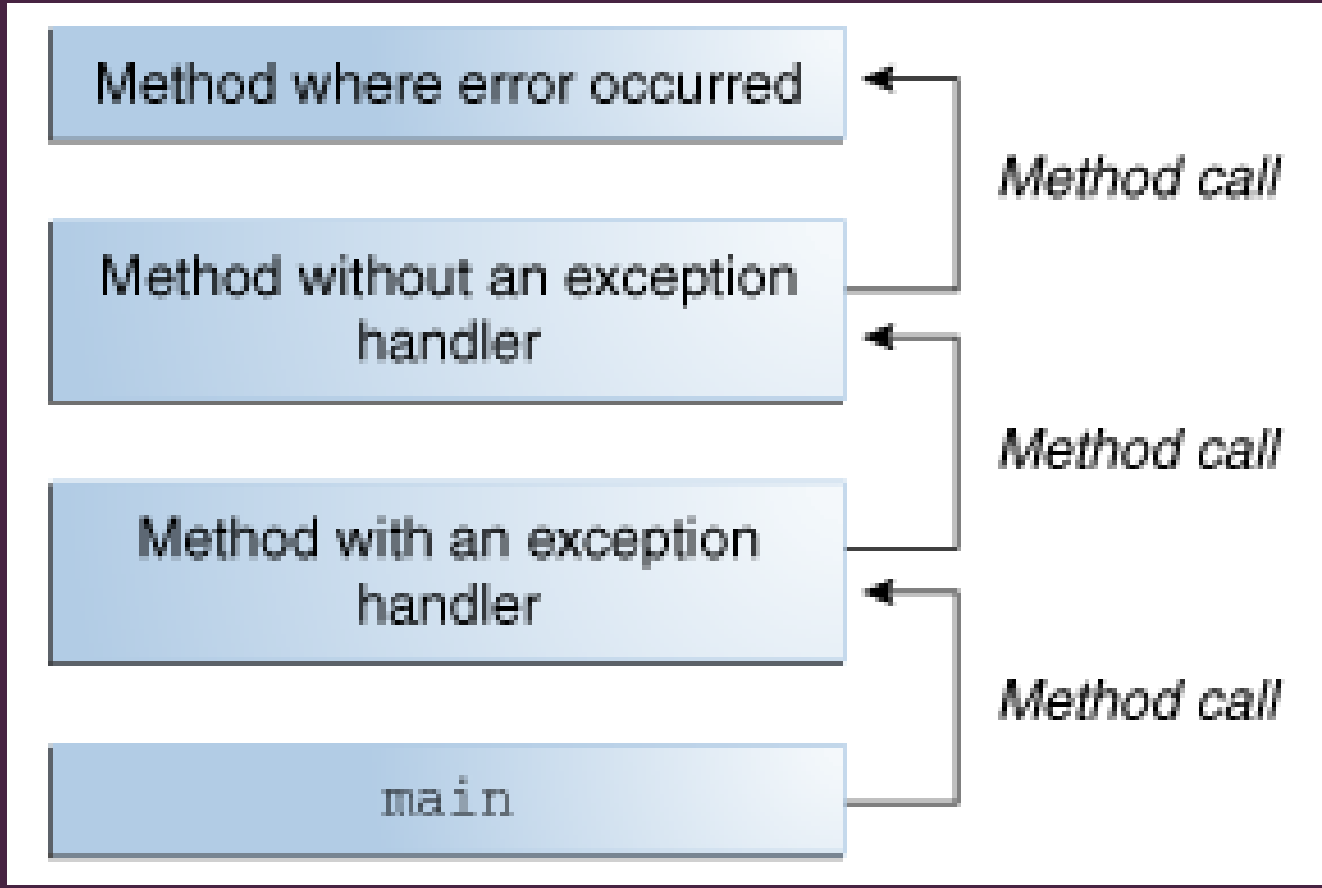
Exception propagation in java

1. When an error occurs within a method, the method creates an object and hands it off to the runtime system. The object, called an exception object, contains information about the error, including its type(DATATYPE) and the state of the program when the error occurred. Creating an exception object and handing it to the runtime system is called throwing an exception.

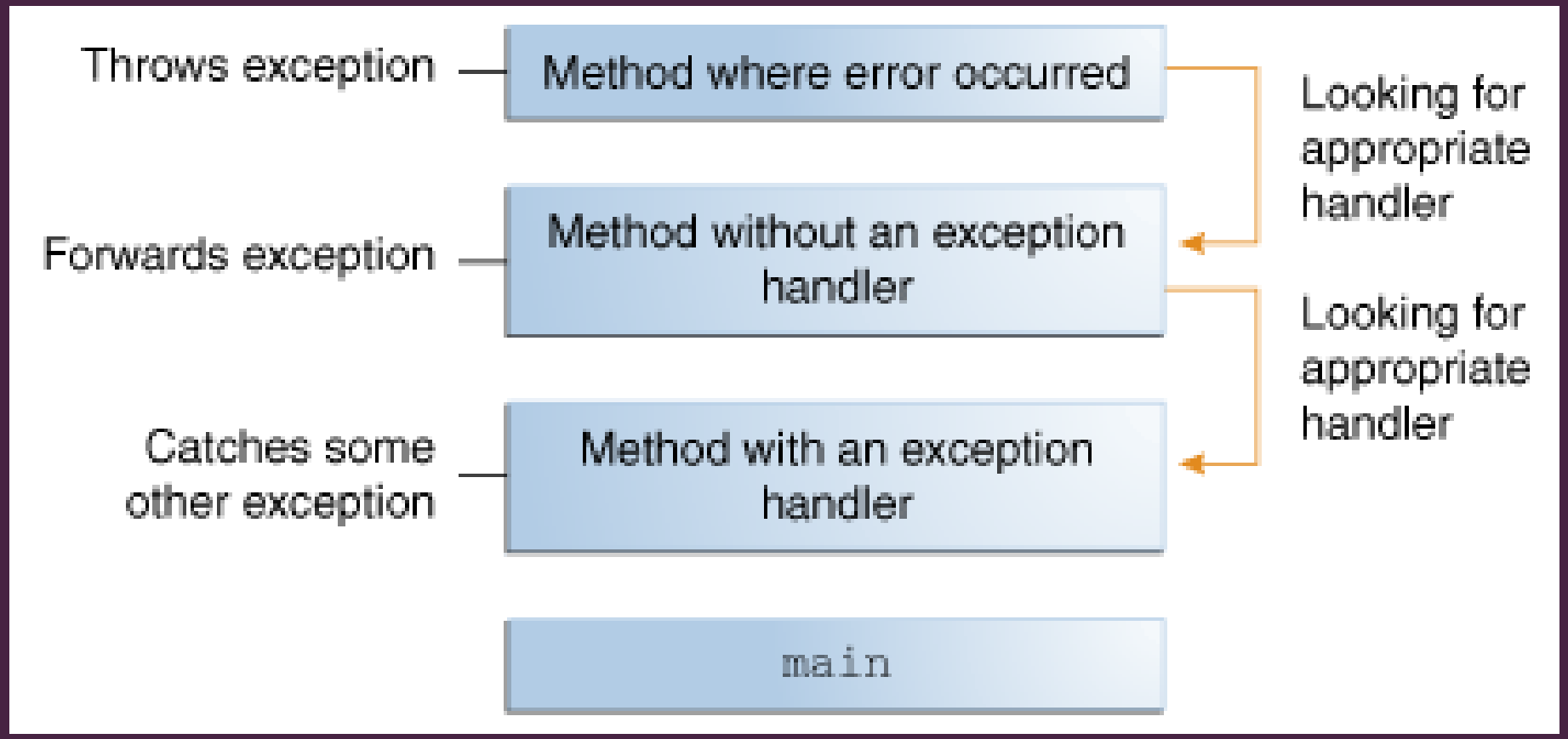
2. After a method throws an exception, the runtime system attempts to find something to handle it. The set of possible "somethings" to handle the exception is the ordered list of methods that had been called to get to the method where the error occurred. The list of methods is known as the call stack

3. The runtime system searches the call stack for a method that contains a block of code that can handle the exception. This block of code is called an exception handler.

4. The search begins with the method in which the error occurred and proceeds through the call stack in the reverse order in which the methods were called. When an appropriate handler is found, the runtime system passes the exception to the handler. An exception handler is considered appropriate if the type of the exception object thrown matches the type that can be handled by the handler.



5. Or the exception handler chosen is said to catch the exception. If the runtime system exhaustively searches all the methods on the call stack without finding an appropriate exception handler, the runtime system (and, consequently, the program) terminates.



Exception propagation

```java
package exceptionpack;
public class TestExceptionPropagation {
    void display() {
        int data;
        data = 50 / 0;
        System.out.println(data);
    }
    void print() {
        display();
    }
    void test() {
        try {
            print();
        } catch (Exception e) {
            System.out.println("exception handled");
        }
    }
    public static void main(String args[]) {
        TestExceptionPropagation obj = new
        TestExceptionPropagation();
        obj.test();
        System.out.println("normal flow...");
    }
}
```

```
exception handled
        normal flow...
```

```java
package exceptionpack;
public class TestExceptionPropagation {
    void display() {
        int data;
        data = 50 / 0;
        System.out.println(data);
    }
    void print() {
        display();
    }
    void test() {
        try {
            print();
        } catch (Exception e) {
            System.out.println("exception handled");
        }
    }
    public static void main(String args[]) {
        TestExceptionPropagation obj = new TestExceptionPropagation();
        obj.test();
        System.out.println("normal flow...");
    }
}
```

Exception occured here

display()

print()

test()

main()

call stack

Java Catch Multiple Exceptions

- A try block can be followed by one or more catch blocks.
- Each catch block must contain a different exception handler.
- So, if you have to perform different tasks at the occurrence of different exceptions, use java multi-catch block.

Points to remember

1. At a time only one exception occurs and at a time only one catch block is executed.
2. All catch blocks must be ordered from most specific to most general, i.e. catch for ArithmeticException must come before catch for Exception.

Multiple Catch

```java
package exceptionpack;

public class MultipleCatchBlock {

    public static void main(String[] args) {
        int ar[] = new int[5];
        try {
            ar[5] = 30 / 0;
        } catch (ArithmeticException ae) {
            System.out.println("Arithmetic Exception occurs");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("ArrayIndexOutOfBounds Exception occurs");
        } catch (Exception e) {
            System.out.println("Parent Exception occurs");
        }
        System.out.println("rest of the code");
    }

}
```

Exception without maintaining the order of exceptions (i.e. from most specific to most general).

```
package exceptiondemos;
public class MultipleCatchBlock {

    public static void main(String[] args){
        try{
            int ar[]=new int[5];
                ar[5]=30/0;
            }catch(Exception ae){
                System.out.println("Arithmetic Exception occurs");
            }catch(ArrayIndexOutOfBoundsException e){
                System.out.println("ArrayIndexOutOfBounds Exception occurs");
            }catch(ArithmeticException e){
                System.out.println("Parent Exception occurs");
        }
        System.out.println("rest of the code");
    }
}
```

```
Compile-time error
```

Important points

```
catch(Exception ae){
    System.out.println("Arithmetic Exception occurs");
}
```

- it is not mandatory to use a generic catch (Exception e) if you're already handling specific exceptions like ArithmeticException and ArrayIndexOutOfBoundsException.

- If you want to catch any other exception that is not Arithmetic or ArrayIndexOutOfBounds, you can add. This acts as a fallback or default handler for all other exceptions.

Nested try block - In Java, using a try block inside another try block is permitted.

Nested Try

```java
package exceptionpack;

public class NestedTryBlock {

    public static void main(String args[]) {
        int data;
        int arr[] = new int[5];


        // outer try block
        try {
            // inner try block 1
            try {
                System.out.println("going to divide by 0");
                data = 39 / 0;
            }catch (ArithmeticException e) {// catch block of inner try block 1
                System.out.println(e);
            }
```

```
        // inner try block 2
        try {
        // assigning the value out of array bounds
            arr[5] = 4;
        } catch (ArrayIndexOutOfBoundsException e) {// catch block of inner try block 2
            System.out.println(e);
        }
        System.out.println("other statement");
    } catch (Exception e) {// catch block of outer try block
        //System.out.println("handled the exception (outer catch)");// no need of this
    statement
    }
    System.out.println("normal flow..");
  }
}
```

Console

```
<terminated> NestedTryBlock [Java Application] C:\Program Files\
going to divide by 0
java.lang.ArithmeticException: / by zero
java.lang.ArrayIndexOutOfBoundsException: 5
other statement
normal flow..
```

finally

1. Java finally block is a block used to execute important code such as closing the connection, etc.
2. Java finally block is always executed whether an exception is handled or not. Therefore, it contains all the necessary statements that need to be printed regardless of the exception occurs or not.
3. The finally block follows the try-catch block or try block.

```
 finally {
System.out.println("finally block is always executed");
}
```

Finally block without exception

```java
package exceptionpack;

public class TestFinallyBlock {

    public static void main(String args[]) {
        int data;
        try {
            data = 25 / 25; // or data=25/10;
            System.out.println(data);
            System.out.println("helloo...");
        } catch (Exception e) {
            System.out.println(e);
        } finally { //// executed regardless of exception occurred or not
            System.out.println("finally block is always executed");
        }
        System.out.println("rest of the code...");
    }


}
```

Finally block with exception

```java
package exceptionpack;

public class TestFinallyBlock {

    public static void main(String args[]) {
        int data;
        try {
            data = 25 / 0; // or data=25/10;
            System.out.println(data);
            System.out.println("helloo...");
        } catch (Exception e) {
            System.out.println(e);
        } finally { //// executed regardless of exception occurred or not
            System.out.println("finally block is always executed");
        }
        System.out.println("rest of the code...");
    }


}
```

Console

```
<terminated> TestFinallyBlock [Java Application] C:\Prog
java.lang.ArithmeticException: / by zero
finally block is always executed
rest of the code...
```

Throw keyword

- The Java throw keyword is used to throw an exception explicitly. i.e throw is a statement used to explicitly throw an exception object at runtime.
- throw can be used with both checked and unchecked exceptions.
- We can also define our own set of conditions and throw an exception msg explicitly using throw keyword. (e.g. **`Person is not eligible to vote`**)
- It is commonly used to throw custom exceptions, but it is not limited to that.
- When you throw a checked exception, it must be either caught or declared using throws.

Throws keyword

1. The Java throws keyword is used to declare an exception. It gives an information to the programmer that there may occur an exception. So, it is better for the programmer to provide the exception handling code so that the normal flow of the program can be maintained.

Throw keyword – bad practice with exception

```java
package exceptionpack;


public class TestThrow {
    // function to check if person is eligible to vote or not
    public static void validate(int age) {
        if (age < 18) {
            // throw Arithmetic exception if not eligible to vote
            throw new ArithmeticException("Person is not eligible to vote");
        } else {
            System.out.println("Person is eligible to vote!!");
        }
    }
    public static void main(String args[]) {// main method
        validate(13);// calling the function
        System.out.println("rest of the code...");
    }


}
```

```
Exception in thread "main" java.lang.ArithmeticException: Person is not eligible to vote
        at oopsprg.TestThrow.validate(TestThrow.java:8)
        at oopsprg.TestThrow.main(TestThrow.java:17)
```

```java
package exceptionpack;


public class TestThrow {
    // function to check if person is eligible to vote or not
    public static void validate(int age) {
        if (age < 18) {
            // throw Arithmetic exception if not eligible to vote
            throw new ArithmeticException("Person is not eligible to vote");
        } else {
            System.out.println("Person is eligible to vote!!");
        }
    }
    public static void main(String args[]) {// main method
        validate(19);// calling the function
        System.out.println("rest of the code...");
    }


}
```

Throw keyword – bad practice without exception

```
Person is eligible to vote!!
rest of the code...
```

Throw keyword – good practice

```java
package exceptionpack;


public class TestThrow {
    // function to check if person is eligible to vote or not
    public static void validate(int age) throws ArithmeticException {
        if (age < 18) {
            // throw Arithmetic exception if not eligible to vote
            throw new ArithmeticException("Person is not eligible to vote");
        } else {
            System.out.println("Person is eligible to vote!!");
        }
    }
    public static void main(String args[]) {// main method
    try {

        validate(13);// calling the function
        }catch(ArithmeticException ae){
            ae.printStackTrace();
        }
        System.out.println("rest of the code...");
    }


}
```

Throws keyword

```java
package exceptionpack;
import java.io.IOException;
public class ThrowThrowsCheckedExample {

    // Method that declares it may throw IOException
    static void processFile() throws IOException {
        // Create and throw a checked exception explicitly
        throw new IOException("File processing failed - custom checked exception");
    }


    public static void main(String[] args) {
        try {
            // Call method that might throw IOException
            processFile();
        } catch (IOException e) {
            System.out.println("Caught checked exception: " + e.getMessage());
        }
    }
}
```

Problems | Javadoc | Declaration | Console

```
<terminated> ThrowThrowsCheckedExample [Java Application] H:\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.9
Caught checked exception: File processing failed - custom checked exception
```

Why throw is actually useful in real-time

The throw keyword is used to deliberately signal a problem when your program detects something invalid or unexpected, even if Java doesn't raise an exception automatically.

In real-time systems like banking, exam, user input, throw is crucial to enforce logic and protect the system.

Eg. 1: Validating User Input

If a user enters invalid data (e.g., age below 0), you can throw a custom or built-in exception.

```
public void setAge(int age) {
    if (age < 0) {
        throw new IllegalArgumentException("Age cannot be negative");
    }
    this.age = age;
}
```

Eg. 2: Business Rule Violation

If some logic in your business model must be strictly followed.

```
public void withdraw(double amount) throws Exception {
    if (amount > balance) {
        throw new Exception("Insufficient balance");
    }
    balance -= amount;
}
```

Eg. 3. Custom Exceptions for Clarity

If your app has a custom rule that a student must score at least 40 to pass:

```
if (marks < 40) {
    throw new FailException("Student failed");
}
```

Why throws is along with throw?

- Throws is just a declaration — it tells the compiler and caller, "Hey, I might throw this exception, so be ready.

- "But without throw, nothing is actually thrown. So you need both

If we throw a checked exception using throw keyword,

1. it is must to handle the exception using catch block
2. or the method must declare it using throws declaration. (e.g. Read values from an existing file. Checked Exception will be occurred if file doesn't exist)

Program 1: Handling Checked Exception using try-catch

```java
package exceptionpack;

import java.io.IOException;

public class HandleCheckedException {

    public static void main(String[] args) {
        try {
            // throwing a checked exception
            throw new IOException("File not found");
        } catch (IOException e) {
            System.out.println("Caught Exception: " + e.getMessage());
        }
    }
}
```

Program 2: Declaring Checked Exception using throws

```java
package exceptionpack;
import java.io.IOException;
public class DeclareCheckedException {


    // method declares that it may throw IOException
    static void readFile() throws IOException {
        throw new IOException("File can't be read");
    }


    public static void main(String[] args) {
        try {
            readFile(); // calling method that throws exception
        } catch (IOException e) {
            System.out.println("Handled in main: " + e.getMessage());
        }
    }
}
```

Throwing User-defined Exception

```java
package exceptionpack;


//class representing custom exception
public class InvalidAgeException extends Exception {
    public InvalidAgeException(String str) {
        // calling the constructor of parent Exception
        super(str);
    }
}
```

```java
package exceptionpack;
//class that uses custom exception InvalidAgeException
public class TestCustomException {
    // method to check the age
    static void validate(int age) throws InvalidAgeException {
        if (age < 18) {
            // throw an object of user defined exception
            throw new InvalidAgeException("age is not valid to vote");
        } else {
            System.out.println("welcome to vote");
        }
    }
    public static void main(String args[]) {
        try {
            // calling the method
            validate(13);
        } catch (InvalidAgeException ex) {
            System.out.println("Caught the exception");
            // printing the message from InvalidAgeException object
            System.out.println("Exception occured: " + ex);
        }
        System.out.println("rest of the code...");
    }
}
```

What is try-with-resources?

- In Java 7, the feature try-with-resources feature, which allows you to avoid writing a finally block when working with resources like files, streams, or sockets.

- try that automatically closes any object that implements the AutoCloseable interface — like FileReader, BufferedReader, Scanner, Connection, etc.

```
try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {
    System.out.println(br.readLine());
} catch (IOException e) {
    e.printStackTrace();
}
// No finally block needed — resource is closed automatically
```

Benefits:

- No more boilerplate finally block.
- Cleaner and safer code.
- Multiple resources can be managed

```
try (FileReader fr = new FileReader("a.txt");
    BufferedReader br = new BufferedReader(fr)) {
   // ...
}
```

Multi-Catch

Rules:

- All exception types must be unrelated (i.e., not in the same inheritance chain).
- The caught exception variable (e) is implicitly final — you cannot assign a new value to it.

```
import java.io.*;

public class MultiCatchExample {
    public static void main(String[] args) {
        String s=null ;
        int num;
        try {
          System.out.println(s.length());   // NullPointerException
            num = 10 / 0;                   // ArithmeticException
        } catch (NullPointerException | ArithmeticException e) {
           System.out.println("Handled: " + e);
        }
    }
}
```

Problems Javadoc Declaration Console

```
<terminated> MultiCatchExample [Java Application] H:\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.9.v20231028-0858\jre\bin\javaw.exe (Jul 2, 2
Handled: java.lang.NullPointerException: Cannot invoke "String.length()" because "s" is null
```

Invalid (not allowed)

Because IOException is a subclass of Exception — Java won't allow overlapping exception types in multi-catch.

```
catch (IOException | Exception e) {  //  Compile-time error
}
```

throw	throws
The Java throw keyword is used to throw an exception explicitly	The Java throws keyword is used to declare an exception
The throw keyword is followed by an instance of Exception to be thrown.	The throws keyword is followed by class names of Exceptions to be thrown.
throw is used within the method.	throws is used with the method signature.
We are allowed to throw only one exception at a time i.e. we cannot throw multiple exceptions.	We can declare multiple exceptions using throws keyword that can be thrown by the method. For example, main() throws IOException, SQLException.

final	finally	finalize
final is the keyword and access modifier which is used to apply restrictions on a class, method or variable.	finally is the block in Java Exception Handling to execute the important code whether the exception occurs or not.	finalize is the method in Java which is used to perform clean up processing just before object is garbage collected.
Final keyword is used with the classes, methods and variables.	Finally block is always related to the try and catch block in exception handling.	finalize() method is used with the objects.
(1) Once declared, final variable becomes constant and cannot be modified. (2) final method cannot be overridden by sub class. (3) final class cannot be inherited.	(1) finally block runs the important code even if exception occurs or not. (2) finally block cleans up all the resources used in try block	finalize method performs the cleaning activities with respect to the object before its destruction.
Final method is executed only when we call it.	Finally block is executed as soon as the try-catch block is executed.It's execution is not dependant on the exception.	finalize method is executed just before the object is destroyed.

Thank you ☺ Happy learning ☺